

Jurnal etapa 2 – Petrenciuc Amelia-Andreea

1. Cerințe rezolvate

În această etapă, s-a realizat separarea și izolarea componentelor sistemului implementat în etapa anterioară folosind tehnologia Docker.

- Dockerizarea Modelului LLM: S-a creat un container dedicat pentru Ollama care rulează modelul llama3.2:latest. S-a asigurat persistența modelelor descărcate și pornirea automată a serviciului.
- Dockerizarea Serverului MCP: S-a migrat serverul MCP de la comunicarea standard I/O (stdio) la protocolul HTTP (SSE - Server-Sent Events) pe portul 8002, permițând accesul prin rețea. S-a creat un container care rulează acest server și montează directorul administrat.
- Dockerizarea Agentului și Interfeței Web: S-a creat un container pentru adk-web care găzduiește agentul SysAdmin_Agent. S-a configurat agentul să comunice cu celelalte două containere prin rețeaua Docker.
- Orchestrare cu Docker Compose: S-a realizat un fișier docker-compose.yml care definește rețeaua aso-network, gestionează dependențele dintre servicii (healthchecks) și volumele necesare.
- Securitate și Networking: S-au configurat corect host-urile (0.0.0.0) pentru a permite comunicarea inter-container și s-au mapat volumele pentru a reflecta fișierele reale de pe sistemul gazdă în mediul izolat.

2. Modul de rezolvare

Arhitectura sistemului a fost modificată pentru a funcționa într-un mediu distribuit (microservicii), unde fiecare componentă are propriul mediu de execuție.

Structura Proiectului: Am reorganizat proiectul într-o structură modulară (os-proiect), având fișiere Dockerfile distincte pentru fiecare serviciu, dar partajând un fișier requirements.txt comun la rădăcină pentru gestionarea dependențelor (google-adk, litellm, fastmcp, uvicorn).

Configurarea Containerelor:

1. Serviciul Ollama:

- Am utilizat imaginea de bază ollama/ollama:latest.
- Am creat un script de start (start.sh) care pornește serverul în fundal, așteaptă ca API-ul să fie disponibil și descarcă automat modelul llama3.2 dacă acesta nu există. Acest lucru asigură că agentul are întotdeauna modelul necesar disponibil la pornire.

2. Serviciul MCP Server:

- S-a modificat codul `server_mcp.py` pentru a inițializa FastMCP cu parametrul `host="0.0.0.0"` și `port=8002`. Aceasta a fost o modificare critică față de rularea locală, deoarece containerele Docker izolează interfața localhost.
- În Dockerfile, s-a asigurat copierea codului sursă și instalarea dependențelor în ordinea corectă.
- Directorul administrat a fost definit prin variabila `MANAGED_ROOT = "/mnt/admin_root"`, care corespunde punctului de montare din container.

3. Serviciul ADK Web (Agent):

- S-a configurat containerul să monteze întreg proiectul `os-project` în `/app`.
- Agentul a fost configurat să se conecteze la URL-urile interne ale rețelei Docker: `http://ollama:11434` pentru LLM și `http://mcp-server:8002/mcp` pentru unelte.

Orchestrarea (Docker Compose): Fișierul `docker-compose.yml` a fost punctul central al acestei etape.

- Rețea: Toate serviciile sunt conectate la `aso-network` (driver bridge).
- Volume:
 - Pentru Ollama: `ollama_data:/root/.ollama` (pentru a nu descărca modelul la fiecare restart).
 - Pentru MCP: `./admin_root:/mnt/admin_root` (pentru a expune fișierele reale de pe disc către agent).
- Variabile de mediu: S-a folosit `ADK_AGENT_MODULE` setat la `"os_project.agent.agent:root_agent"` pentru a instrui serverul ADK exact unde să găsească instanța agentului în structura de pachete Python.

3. Probleme întâmpinate și modul de rezolvare

Această etapă a prezentat provocări semnificative legate de modul în care Docker gestionează fișierele și rețeaua.

- Eroarea "Directory does not exist" (Volume Mapping):
 - *Simptom:* Agentul răspundea corect la comenzi, dar când îi ceream să listeze fișierele din `./lab4`, spunea că directorul nu există, deși el exista pe PC-ul meu.
 - *Cauză:* În `docker-compose.yml`, calea volumului era greșită (`./mcp-server/admin_root:/mnt/admin_root`). Docker nu găsea folderul la acea cale relativă și, implicit, crea un director gol în container.
 - *Rezolvare:* Am corectat calea volumului la `./admin_root:/mnt/admin_root`, asigurând corespondența corectă între fișierele de pe Windows și calea `/mnt/admin_root` din Linux (container).
- Eroarea "Connection Refused" la MCP:
 - *Simptom:* Agentul nu se putea conecta la serverul MCP.

- *Cauză:* Serverul MCP asculta implicit pe 127.0.0.1 (localhost). Într-un container, localhost este izolat de exterior.
- *Rezolvare:* Am modificat instanțierea FastMCP în server_mcp.py adăugând `host="0.0.0.0"`, permițând conexiuni de la orice interfață de rețea din container.
- Eroare la construirea imaginii (Dockerfile order):
 - *Simptom:* COPY requirements.txt eșua sau pip install nu găsea fișierul.
 - *Rezolvare:* Am corectat ordinea comenzilor în Dockerfile, asigurându-mă că fișierul requirements.txt este copiat înainte de a rula comanda RUN pip install.

4. Concluzii

În Etapa 2, am reușit să transform un script local simplu într-o arhitectură de microservicii robustă și portabilă. Am învățat cum să scriu fișiere Dockerfile optimizate, cum să gestionez dependențele Python într-un mediu containerizat și, cel mai important, cum să rezolv problemele de networking și volume mapping specifice Docker.

Sistemul rezultat este acum capabil să ruleze pe orice mașină cu Docker instalat, având componentele (LLM, Logică, Interfață) complet decuplate și scalabile. Agentul poate acum să inspecteze și să administreze fișiere reale de pe sistemul gazdă, dar într-un mediu controlat și izolat.

TEST

